1 Data representation

In this chapter you will learn about:

- ★ number systems
  - how and why computers use binary to represent data
  - the denary, binary and hexadecimal number systems
  - converting numbers between denary, binary and hexadecimal
  - how and why hexadecimal is used for data representation
  - how to add two positive 8-bit numbers
  - overflow when performing binary addition
  - logical binary shifts on positive 8-bit integers
  - two's complement notation to represent positive and negative binary numbers
- ★ text, sound and images
  - how and why a computer represents text
  - the use of character sets including ASCII and Unicode
  - how and why a computer represents sound
  - sound sample rate and sample resolution
  - how and why a computer represents an image
  - the effects of the resolution and colour depth on images
- ★ data storage and compression
  - how data storage is measured
  - calculating the file size of an image and sound file
  - the purpose of and need for data compression
  - lossy and lossless compression.

This chapter considers the three key number systems used in computer science, namely binary, denary and hexadecimal. It also discusses how these number systems are used to measure the size of computer memories and storage devices, together with how sound and images can be represented digitally.

1.1 Number systems

1.1.1 Binary represents data

As you progress through this book you will begin to realise how complex computer systems really are. By the time you reach Chapter 10 you should have a better understanding of the fundamentals behind computers themselves and the software that controls them.

You will learn that any form of data needs to be converted into a binary format so that it can be processed by the computer.

However, no matter how complex the system, the basic building block in all computers is the binary number system. This system is chosen because it only consists of 1s and 0s. Since computers contain millions and millions of tiny 'switches', which must be in the ON or OFF position, they can be represented by the binary system. A switch in the ON position is represented by 1; a switch in the OFF position is represented by 0.

Switches used in a computer make use of logic gates (see Chapter 10) and are used to store and process data.

1.1.2 Binary, denary and hexadecimal systems

The binary system

We are all familiar with the denary number system which counts in multiples of 10. This gives us the well-known headings of units, 10s, 100s, 1000s, and so on:

(10^4)	(10^3)	(10^2)	(10^1)	(10^0)
10 000	**1000**	**100**	**10**	**1**
2	5	1	7	7

Denary uses ten separate digits, 0-9, to represent all values. Denary is known as a base 10 number system.

The **binary number system** is a base 2 number system. It is based on the number 2. Thus, only the two 'values' 0 and 1 can be used in this system to represent all values. Using the same method as denary, this gives the headings 2^0, 2^1, 2^2, 2^3, and so on. The typical headings for a binary number with eight digits would be:

(2^7)	(2^6)	(2^5)	(2^4)	(2^3)	(2^2)	(2^1)	(2^0)
128	**64**	**32**	**16**	**8**	**4**	**2**	**1**
1	1	1	0	1	1	1	0

A typical binary number would be: 11101110.

Converting from binary to denary

The conversion from binary to denary is a relatively straightforward process. Each time a 1-value appears in a binary number column, the column value (heading) is added to a total. This is best shown by three examples which use 8-bit, 12-bit and 16-bit binary numbers:

Example 1

Convert the binary number, 11101110, into a denary number.

128	64	32	16	8	4	2	1
1	1	1	0	1	1	1	0

The equivalent denary number is 128 + 64 + 32 + 8 + 4 + 2 = 238

Example 2

Convert the following binary number, 011110001011, into a denary number.

2048	1024	512	256	128	64	32	16	8	4	2	1
0	1	1	1	1	0	0	0	1	0	1	1

The equivalent denary number is 1024 + 512 + 256 + 128 + 8 + 2 + 1 = 1931

Activity 1.2

Convert the following denary numbers into binary (using both methods):

a 41	**d** 100	**g** 144	**j** 255	**m** 4095
b 67	**e** 111	**h** 189	**k** 33000	**n** 16400
c 86	**f** 127	**i** 200	**l** 888	**o** 62307

The hexadecimal system

The **hexadecimal number system** is very closely related to the binary system. Hexadecimal (sometimes referred to as simply 'hex') is a base 16 system and therefore needs to use 16 different 'digits' to represent each value.

Because it is a system based on 16 different digits, the numbers 0 to 9 and the letters A to F are used to represent each hexadecimal (hex) digit. A in hex = 10 in denary, B = 11, C = 12, D = 13, E = 14 and F = 15.

Using the same method as for denary and binary, this gives the headings 16^0, 16^1, 16^2, 16^3, and so on. The typical headings for a hexadecimal number with five digits would be:

(16^4)	(16^3)	(16^2)	(16^1)	(16^0)
65 536	**4096**	**256**	**16**	**1**
2	1	F	3	A

A typical example of hex is 2 1 F 3 A.

Since $16 = 2^4$ this means that FOUR binary digits are equivalent to each hexadecimal digit. The following table summarises the link between binary, hexadecimal and denary:

▼ **Table 1.1**

Binary value	Hexadecimal value	Denary value
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15